

Roteiro do vídeo: Pipeline de treinamento de deep learning: COVID-19 em tomografias

Item	Detalhe
Notebook	pipeline_de_agentes.ipynb (executado, com todas as saídas)
Cenas	31, cada uma com uma tela capturada do Jupyter (1920×1080, pasta telas/)
Narração	1285 palavras, em tom de aula
Duração estimada	9 min 06 s (~150 palavras/min + 1 s de pausa por cena)
Fonte para montar o vídeo	roteiro.json (tela, narração, início e duração de cada cena)
Revisão	Todas as telas foram conferidas contra a narração (números, nomes e trechos citados)

Roteiro cena a cena

Cena 01 Abertura

0:00 → 0:21 · 21 s



Na tela: Título, objetivo e o fluxo do pipeline telas/01_abertura.png

Olá! Nesta aula vamos construir um pipeline completo de treinamento de um modelo de deep learning. O problema é real: olhando uma tomografia computadorizada do pulmão, o modelo deve dizer se o caso é positivo para COVID-19 ou normal. Um aviso importante: é um exemplo didático, não serve para diagnóstico médico.



Na tela: Fluxo em 7 etapas e a tabela de conceitos telas/02_pipeline_conceitos.png

O caminho é o de qualquer projeto de visão computacional: dividir as imagens, carregá-las, aproveitar um modelo pré-treinado, treinar, avaliar e fazer previsões. Deep learning são redes neurais com muitas camadas que aprendem padrões direto dos pixels. E transfer learning é reaproveitar uma rede já treinada em milhões de imagens para um problema com poucas imagens, como o nosso.

1. Bibliotecas e verificação da GPU

Usamos três grupos de ferramentas:

- **TensorFlow / Keras:** a biblioteca de deep learning. O Keras é a "camada amigável" do TensorFlow, onde montamos e treinamos as redes.
- **scikit-learn:** métricas de avaliação (matriz de confusão, relatório de classificação).
- **matplotlib:** gráficos e visualização das imagens.

Treinar redes neurais envolve milhões de multiplicações. A **GPU** (placa de vídeo) faz essas contas em paralelo e acelera o treino em dezenas de vezes. A última linha mostra se o TensorFlow encontrou uma GPU.

```
keras.utils.set_random_seed(42) fixa a "semente" dos números aleatórios, para que os resultados sejam parecidos a cada execução.
```

```
[2]: import os, random, shutil, time, warnings
from pathlib import Path

os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3" # esconde as mensagens internas do TensorFlow
os.environ["TF_ENABLE_ONEDNN_OPTS"] = "0" # esconde avisos que não interessam para a aula
warnings.filterwarnings("ignore")

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import keras
from keras import layers
from keras.applications import VGG16
from keras.applications.vgg16 import preprocess_input
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

keras.utils.set_random_seed(42)

print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
gpus = tf.config.list_physical_devices("GPU")
print("GPU disponível:", [tf.config.experimental.get_device_details(g)["device_name"] for g in gpus] or "nenhuma (vai rodar na CPU, bem mais")

TensorFlow: 2.11.0 | Keras: 3.15.1
GPU disponível: ['NVIDIA GeForce RTX 5080']
```

2. Configurações de experimento

Na tela: Importações e a saída 'GPU disponível: RTX 5080' telas/03_bibliotecas_gpu.png

Usamos o TensorFlow com o Keras, a biblioteca de deep learning; o scikit-learn, para as métricas; e o matplotlib, para os gráficos. Treinar uma rede envolve milhões de multiplicações, e a GPU, a placa de vídeo, faz essas contas em paralelo. Aqui ela foi encontrada: uma RTX 5080. No Colab, a GPU T4 gratuita também dá conta.

2. Configurações do experimento

Todos os parâmetros ficam reunidos numa célula só, como no arquivo `config.py` dos projetos originais. Assim fica fácil experimentar.

Parâmetro	Significado
PASTA_ORIGINAL	Onde estão as tomografias originais, uma subpasta por classe
PASTA_DIVISAO	Onde vamos criar as pastas de treino, validação e teste
TREINO_FRACAO / VALIDACAO_FRACAO	80% das imagens para treino e 20% para teste; 10% do treino vira validação
TAMANHO_IMAGEM	A VGG16 foi criada para imagens de 224×224 pixels
LOTE (batch)	Quantas imagens a rede vê antes de cada ajuste dos pesos
EPOCAS *	Uma época é uma passada completa por todas as imagens de treino. Estes são limites máximos: o treino para antes se parar de melhorar
TAXA *	A taxa de aprendizado : o tamanho do "passo" em cada ajuste dos pesos

```
[3]: PASTA_ORIGINAL = Path("dados/datasetct")
PASTA_DIVISAO = Path("dados/divisao")
PASTA_MODELOS = Path("modelos")

CLASSES = ["covid", "normal"] # índice 0 = covid (classe positiva), índice 1 = normal

TREINO_FRACAO = 0.80 # 80% treino (+ validação) / 20% teste
VALIDACAO_FRACAO = 0.10 # 10% do treino vira validação

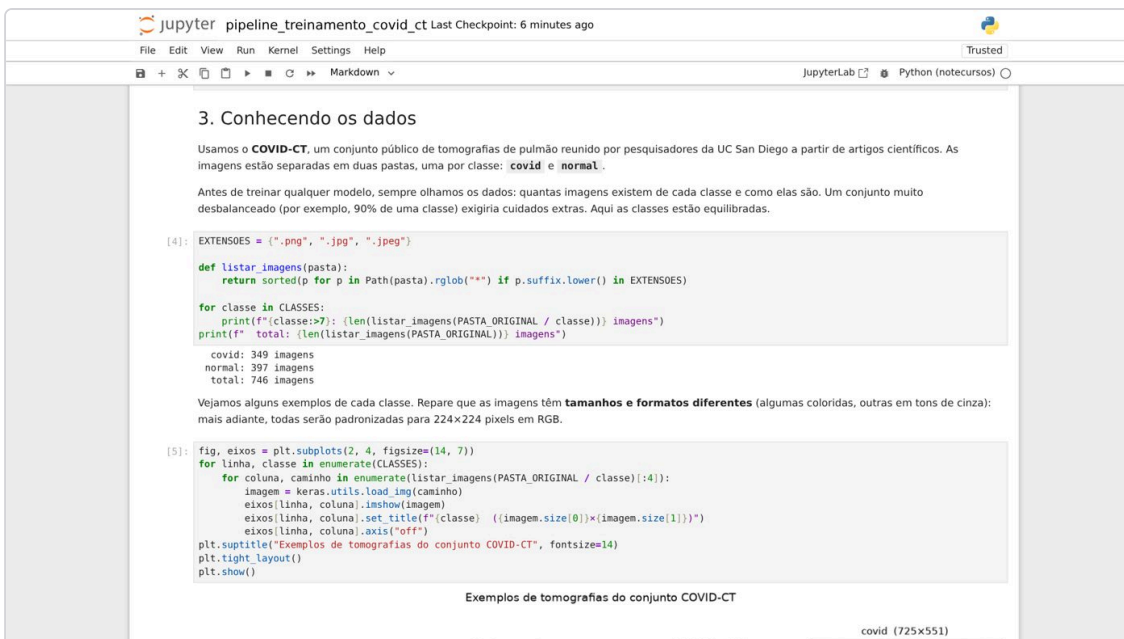
TAMANHO_IMAGEM = (224, 224)
LOTE = 8

EPOCAS_FASE1 = 60 # transfer learning (limite máximo)
EPOCAS_FASE2 = 20 # ajuste fino (limite máximo)
TAXA_FASE1 = 1e-4
TAXA_FASE2 = 1e-5
```

3. Conhecendo os dados

Na tela: Tabela de parâmetros e a célula de configuração telas/04_configuracoes.png

Todos os parâmetros ficam juntos numa célula. Três merecem atenção. O lote é quantas imagens a rede vê antes de cada ajuste; aqui, oito. Uma época é uma passada completa por todas as imagens de treino. E a taxa de aprendizado é o tamanho do passo em cada ajuste dos pesos.



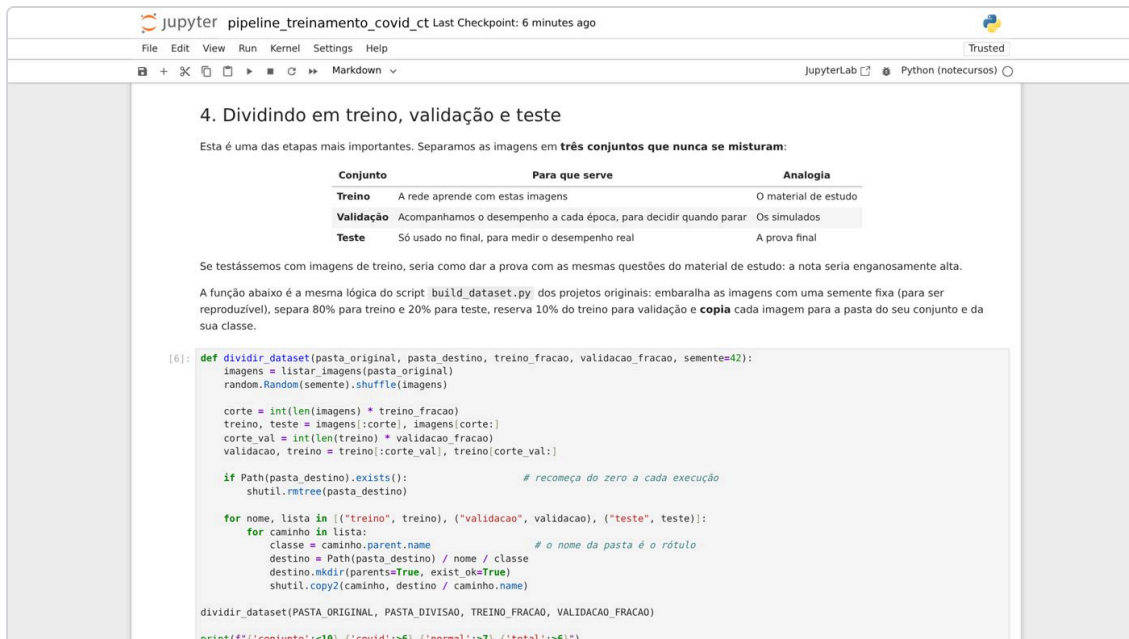
Na tela: Contagem: 349 covid, 397 normal, 746 no total telas/05_dados_contagem.png

Os dados são o COVID-CT, um conjunto público de tomografias. Antes de treinar, sempre olhamos os dados: são 349 imagens de covid e 397 normais, 746 no total. As classes estão equilibradas, o que facilita o treino.



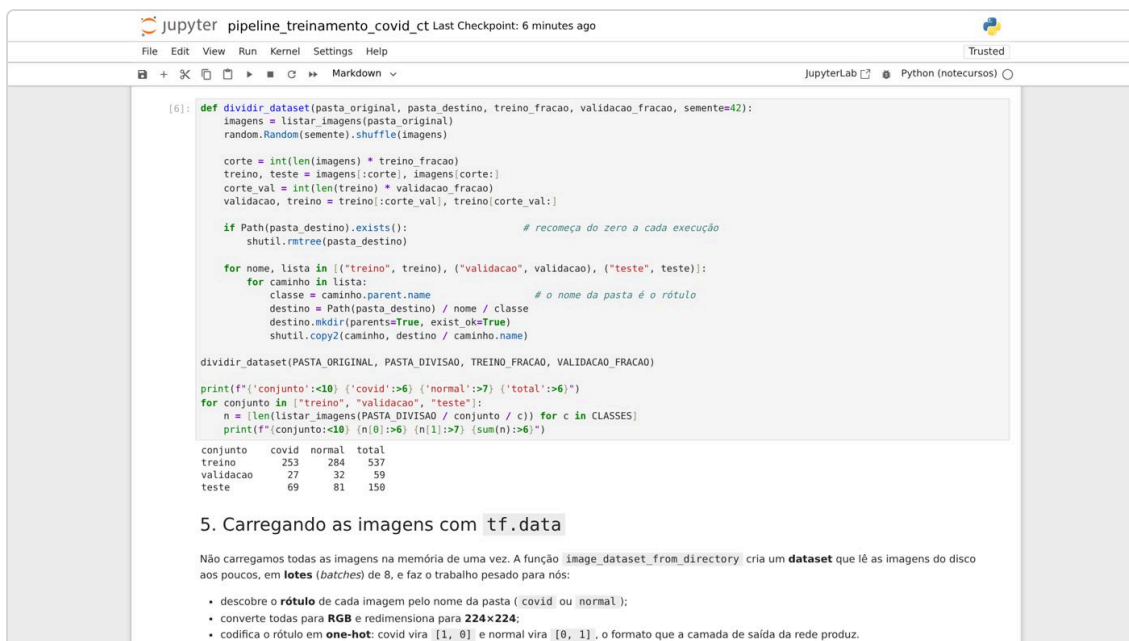
Na tela: Grade com 4 imagens de cada classe telas/06_dados_exemplos.png

Aqui estão alguns exemplos. Repare que as imagens têm tamanhos e formatos diferentes: algumas coloridas, outras em tons de cinza. Mais adiante, todas serão padronizadas para 224 por 224 pixels.



Na tela: Tabela dos três conjuntos com a analogia `telas/07_divisao_explicacao.png`

Esta é uma das etapas mais importantes. Separamos as imagens em três conjuntos que nunca se misturam: o treino, que é o material de estudo; a validação, que funciona como os simulados; e o teste, que é a prova final. Testar com imagens de treino seria dar a prova com as mesmas questões do material de estudo.



Na tela: Função `dividir_dataset` e a tabela 537 / 59 / 150 `telas/08_divisao_codigo.png`

A função embaralha as imagens com uma semente fixa, para ser reproduzível, e copia cada uma para a pasta do seu conjunto e da sua classe. O resultado: 537 imagens de treino, 59 de validação e 150 de teste, sempre com as duas classes.


```

[7]: opcoes = dict(image_size=TAMANHO_IMAGEM, batch_size=LOTE, label_mode="categorical",
                  class_names=CLASSES, color_mode="rgb")

treino_bruto = keras.utils.image_dataset_from_directory(PASTA_DIVISAO / "treino", shuffle=True, seed=42, **opcoes)
validacao_bruto = keras.utils.image_dataset_from_directory(PASTA_DIVISAO / "validacao", shuffle=False, **opcoes)
teste_bruto = keras.utils.image_dataset_from_directory(PASTA_DIVISAO / "teste", shuffle=False, **opcoes)

imagens, rotulos = next(iter(treino_bruto))
print("Formato de um lote de imagens:", imagens.shape, "-> (lote, altura, largura, canais RGB)")
print("Rótulos do lote (one-hot):", rotulos.numpy().astype(int).tolist())

Found 537 files belonging to 2 classes.
Found 59 files belonging to 2 classes.
Found 150 files belonging to 2 classes.
Formato de um lote de imagens: (8, 224, 224, 3) -> (lote, altura, largura, canais RGB)
Rótulos do lote (one-hot): [[1, 0], [1, 0], [1, 0], [0, 1], [1, 0], [1, 0], [0, 1], [1, 0], [1, 0]]

```

5. Carregando as imagens com tf.data

Não carregamos todas as imagens na memória de uma vez. A função `image_dataset_from_directory` cria um **dataset** que lê as imagens do disco aos poucos, em **lotes** (*batches*) de 8, e faz o trabalho pesado para nós:

- descobre o **rótulo** de cada imagem pelo nome da pasta (`covid` ou `normal`);
- converte todas para **RGB** e redimensiona para **224x224**;
- codifica o rótulo em **one-hot**: covid vira `[1, 0]` e normal vira `[0, 1]`, o formato que a camada de saída da rede produz.

Se o conjunto de treino é embaralhado (`shuffle=True`). Validação e teste mantêm a ordem, para compararmos as previsões com os arquivos depois.

6. Aumento de dados e normalização

Aumento de dados (*data augmentation*): com poucas centenas de imagens, a rede corre o risco de **decorar** o conjunto de treino em vez de aprender padrões gerais (o chamado *overfitting*). Para evitar isso, a cada época mostramos versões **levemente modificadas** de cada imagem: um pouco giradas, espelhadas ou aproximadas. Para a rede, é como se tivéssemos mais imagens. Isso vale **só para o treino**.

Normalização: a VGG16 foi treinada no ImageNet com as imagens num formato específico (canais na ordem BGR e a média de cada canal subtraída). A função `preprocess_input` aplica exatamente essa transformação. **Esquecer esse passo** é um erro comum, e os notebooks originais deste projeto não o aplicavam, o que prejudicava os resultados.

Na tela: Explicação de `image_dataset_from_directory` telas/09_carregamento.png

Em vez de carregar tudo na memória, criamos um dataset que lê as imagens do disco em lotes. Ele descobre o rótulo pelo nome da pasta, redimensiona para 224 por 224 e codifica o rótulo em one-hot: covid vira um-zero, e normal, zero-um.

```

[8]: aumento = keras.Sequential([
    layers.RandomRotation(15 / 360), # gira até ±15 graus
    layers.RandomFlip("horizontal"), # espelha horizontalmente
    layers.RandomZoom(0.1), # aproxima/afasta até 10%
], name="aumento_de_dados")

AUTOTUNE = tf.data.AUTOTUNE
treino = treino_bruto.map(lambda x, y: (preprocess_input(aumento(x, training=True)), y)).prefetch(AUTOTUNE)
validacao = validacao_bruto.map(lambda x, y: (preprocess_input(x), y)).prefetch(AUTOTUNE)
teste = teste_bruto.map(lambda x, y: (preprocess_input(x), y)).prefetch(AUTOTUNE)

Veja o aumento de dados em ação: a mesma tomografia, transformada 7 vezes de forma aleatória.

[9]: exemplo = imagens[0:1]
fig, eixos = plt.subplots(1, 8, figsize=(10, 3))
eixos[0].imshow(exemplo[0].numpy().astype("uint8")); eixos[0].set_title("original")
for i in range(1, 8):
    eixos[i].imshow(aumento(exemplo, training=True)[0].numpy().clip(0, 255).astype("uint8"))
    eixos[i].set_title(f"variação {i}")
for eixo in eixos: eixo.axis("off")
plt.tight_layout()

```

6. Aumento de dados e normalização

Aumento de dados (*data augmentation*): com poucas centenas de imagens, a rede corre o risco de **decorar** o conjunto de treino em vez de aprender padrões gerais (o chamado *overfitting*). Para evitar isso, a cada época mostramos versões **levemente modificadas** de cada imagem: um pouco giradas, espelhadas ou aproximadas. Para a rede, é como se tivéssemos mais imagens. Isso vale **só para o treino**.

Normalização: a VGG16 foi treinada no ImageNet com as imagens num formato específico (canais na ordem BGR e a média de cada canal subtraída). A função `preprocess_input` aplica exatamente essa transformação. **Esquecer esse passo** é um erro comum, e os notebooks originais deste projeto não o aplicavam, o que prejudicava os resultados.

O `prefetch` prepara o próximo lote enquanto a GPU processa o atual, para a GPU não ficar esperando.

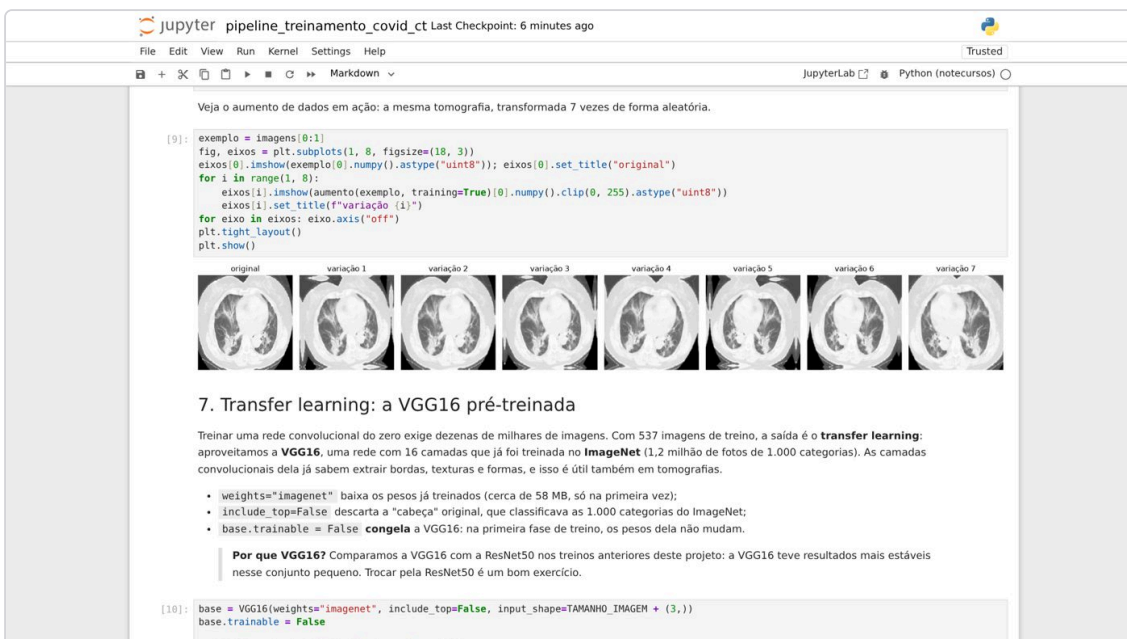
Na tela: Formato (8, 224, 224, 3) e rótulos one-hot telas/10_carregamento_saida.png

A saída confirma: cada lote tem oito imagens de 224 por 224 pixels, com três canais de cor. E aqui estão os oito rótulos, já no formato one-hot.



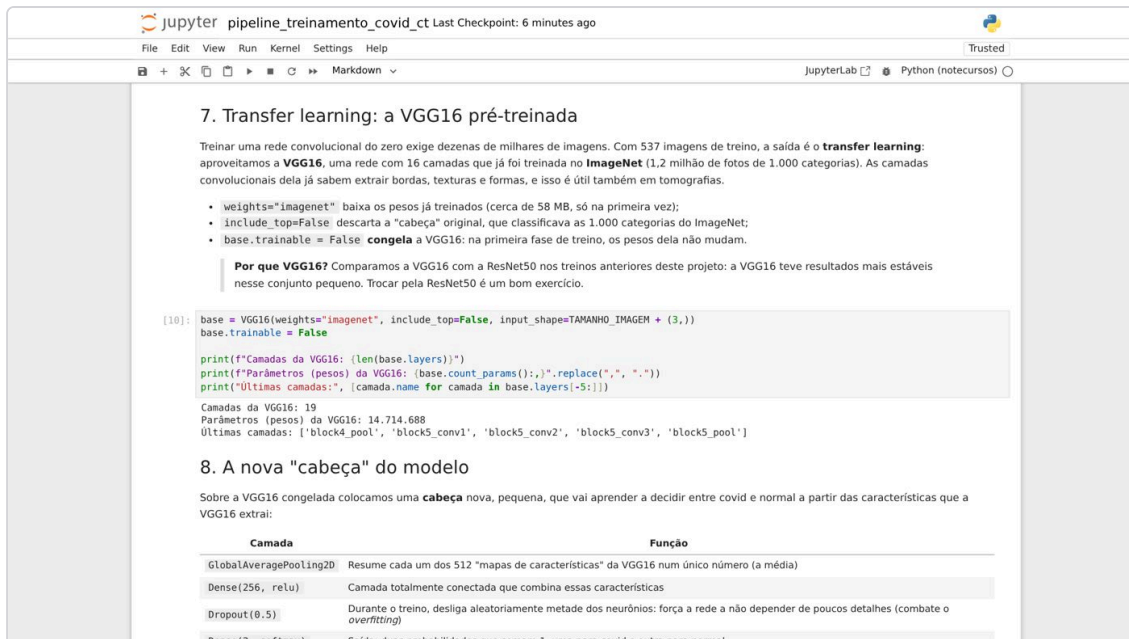
Na tela: Explicação e o código do aumento de dados `telas/11_aumento_explicacao.png`

Com poucas imagens, a rede pode decorar o treino em vez de aprender, o chamado **overfitting**. O aumento de dados mostra, a cada época, versões levemente giradas, espelhadas ou aproximadas das imagens. E a normalização, com o `preprocess_input`, coloca as imagens no formato em que a VGG16 foi treinada. Esquecer esse passo é um erro comum, e os notebooks originais deste projeto o esqueciam.



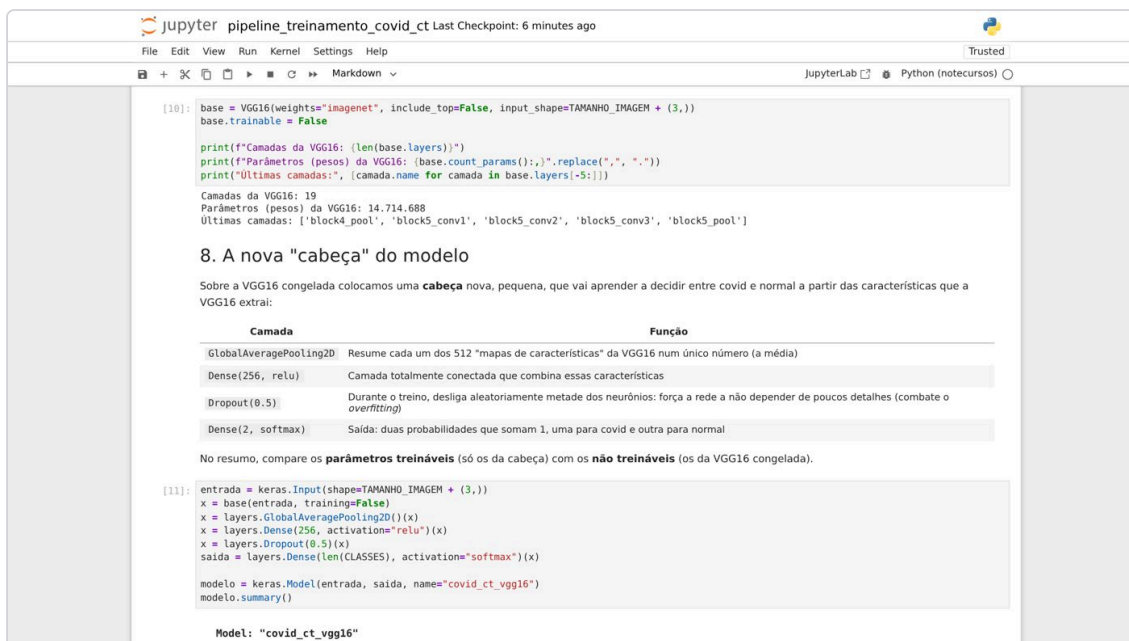
Na tela: A mesma tomografia em 7 variações `telas/12_aumento_exemplo.png`

Veja o efeito: a mesma tomografia, transformada sete vezes de forma aleatória. Para a rede, é como se tivéssemos muito mais imagens.



Na tela: Explicação da VGG16 e dos parâmetros telas/13_vgg16_explicacao.png

Treinar uma rede do zero exigiria dezenas de milhares de imagens. Então usamos a VGG16, uma rede de 16 camadas já treinada no ImageNet, com mais de um milhão de fotos. Descartamos a parte final dela, que classificava mil categorias, e congelamos o resto, para que seus pesos não mudem na primeira fase.



Na tela: Saída: 19 camadas, 14.714.688 parâmetros telas/14_vgg16_saida.png

A VGG16 carregada tem 19 camadas e cerca de 14,7 milhões de parâmetros, que são os pesos aprendidos no ImageNet. As últimas camadas formam o bloco cinco, que vamos usar mais adiante.

8. A nova "cabeça" do modelo

Sobre a VGG16 congelada colocamos uma **cabeça** nova, pequena, que vai aprender a decidir entre covid e normal a partir das características que a VGG16 extrai:

Camada	Função
GlobalAveragePooling2D	Resume cada um dos 512 "mapas de características" da VGG16 num único número (a média)
Dense(256, relu)	Camada totalmente conectada que combina essas características
Dropout(0.5)	Durante o treino, desliga aleatoriamente metade dos neurônios: força a rede a não depender de poucos detalhes (combate o overfitting)
Dense(2, softmax)	Saída: duas probabilidades que somam 1, uma para covid e outra para normal

No resumo, compare os **parâmetros treináveis** (só os da cabeça) com os **não treináveis** (os da VGG16 congelada).

```
[11]: entrada = keras.Input(shape=TAMANHO_IMAGEM + (3,))
x = base(entrada, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)
saida = layers.Dense(len(CLASSES), activation="softmax")(x)

modelo = keras.Model(entrada, saida, name="covid_ct_vgg16")
modelo.summary()
```

Model: "covid_ct_vgg16"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0

Na tela: Tabela das camadas da cabeça `telas/15_cabeça_explicacao.png`

Sobre a VGG16 colocamos uma cabeça nova. O pooling resume as características extraídas; a camada densa as combina; o dropout desliga metade dos neurônios durante o treino, para a rede não depender de poucos detalhes; e a saída softmax devolve duas probabilidades que somam um: covid e normal.

Model: "covid_ct_vgg16"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 2)	514

Total params: 14,846,530 (56.64 MB)

Trainable params: 131,842 (515.01 KB)

Non-trainable params: 14,714,688 (56.13 MB)

9. Treinamento, fase 1: só a cabeça aprende

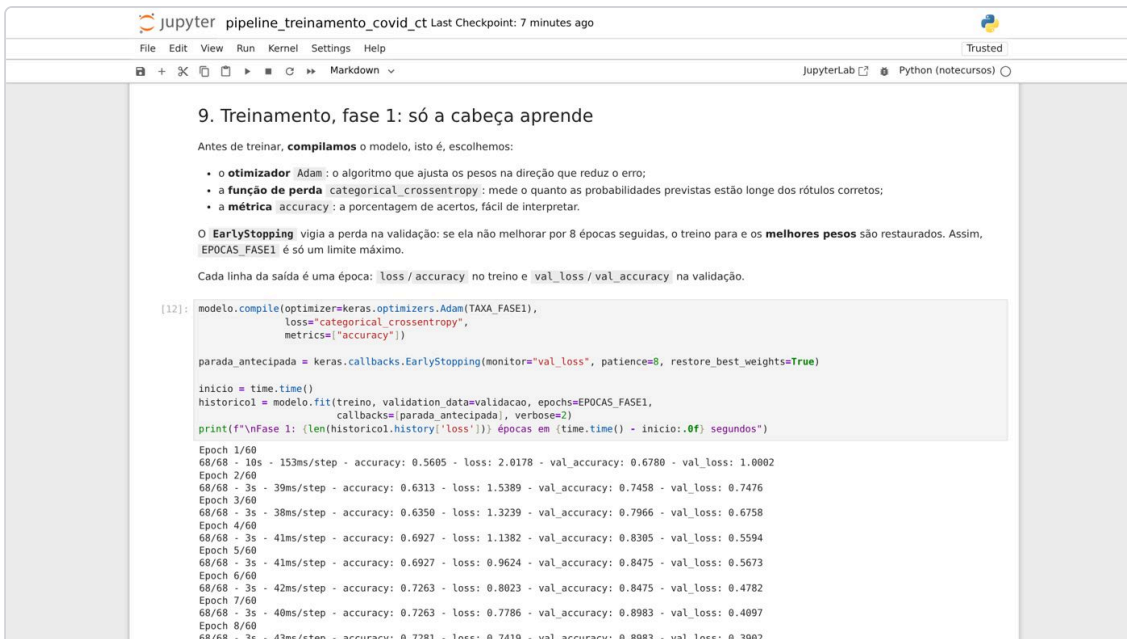
Antes de treinar, **compilamos** o modelo, isto é, escolhemos:

- o **otimizador** Adam: o algoritmo que ajusta os pesos na direção que reduz o erro;
- a **função de perda** `categorical_crossentropy`: mede o quanto as probabilidades previstas estão longe dos rótulos corretos;
- a **métrica** `accuracy`: a porcentagem de acertos, fácil de interpretar.

O **EarlyStopping** vigia a perda na validação: se ela não melhorar por 8 épocas seguidas, o treino para e os **melhores pesos** são restaurados. Assim,

Na tela: Parâmetros treináveis (131.842) e não treináveis `telas/16_cabeça_resumo.png`

No resumo do modelo, compare os números: só cerca de 132 mil parâmetros são treináveis, os da cabeça nova. Os 14,7 milhões da VGG16 estão congelados. É por isso que o transfer learning funciona com poucas imagens.



```
[12]: modelo.compile(optimizer=keras.optimizers.Adam(TAXA_FASE1),
                loss="categorical_crossentropy",
                metrics=["accuracy"])

parada_anticipada = keras.callbacks.EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True)

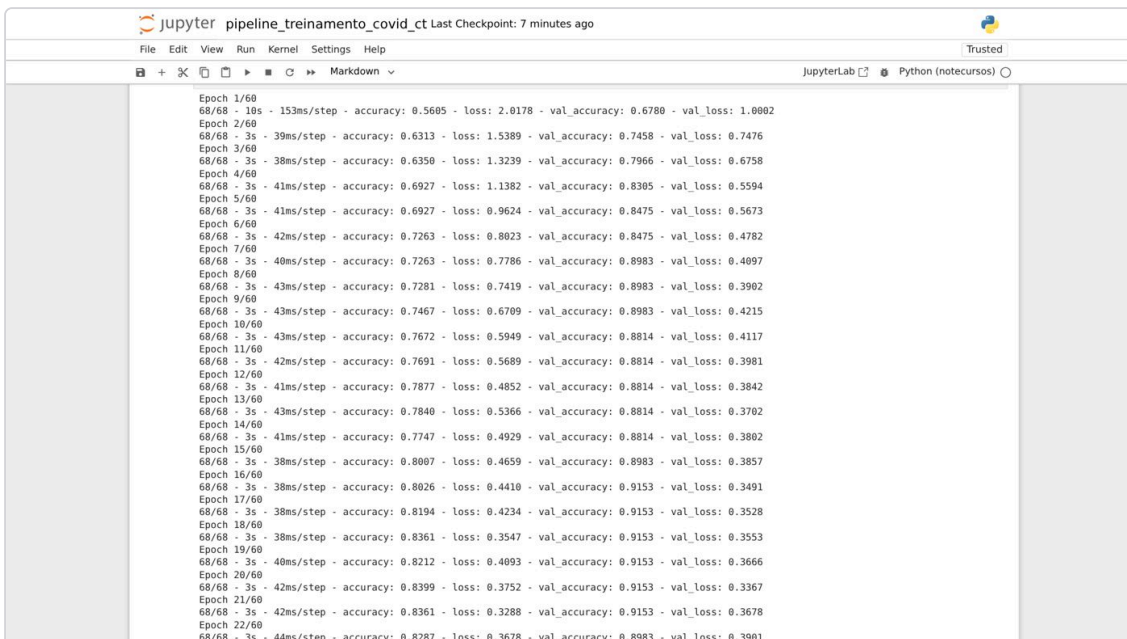
inicio = time.time()
historico1 = modelo.fit(treino, validation_data=validacao, epochs=EPOCHAS_FASE1,
                      callbacks=[parada_anticipada], verbose=2)
print(f"*\nFase 1: {len(historico1.history['loss'])} épocas em {time.time() - inicio:.0f} segundos")

Epoch 1/60
68/68 - 10s - 153ms/step - accuracy: 0.5605 - loss: 2.0178 - val_accuracy: 0.6780 - val_loss: 1.0002
Epoch 2/60
68/68 - 3s - 39ms/step - accuracy: 0.6313 - loss: 1.5389 - val_accuracy: 0.7458 - val_loss: 0.7476
Epoch 3/60
68/68 - 3s - 38ms/step - accuracy: 0.6350 - loss: 1.3239 - val_accuracy: 0.7966 - val_loss: 0.6758
Epoch 4/60
68/68 - 3s - 41ms/step - accuracy: 0.6927 - loss: 1.1382 - val_accuracy: 0.8305 - val_loss: 0.5594
Epoch 5/60
68/68 - 3s - 41ms/step - accuracy: 0.6927 - loss: 0.9624 - val_accuracy: 0.8475 - val_loss: 0.5673
Epoch 6/60
68/68 - 3s - 42ms/step - accuracy: 0.7263 - loss: 0.8023 - val_accuracy: 0.8475 - val_loss: 0.4782
Epoch 7/60
68/68 - 3s - 40ms/step - accuracy: 0.7263 - loss: 0.7786 - val_accuracy: 0.8983 - val_loss: 0.4097
Epoch 8/60
68/68 - 3s - 43ms/step - accuracy: 0.7281 - loss: 0.7419 - val_accuracy: 0.8983 - val_loss: 0.3902
```

Na tela: compile: Adam, categorical_crossentropy, accuracy e EarlyStopping

telas/17_treino1_explicacao.png

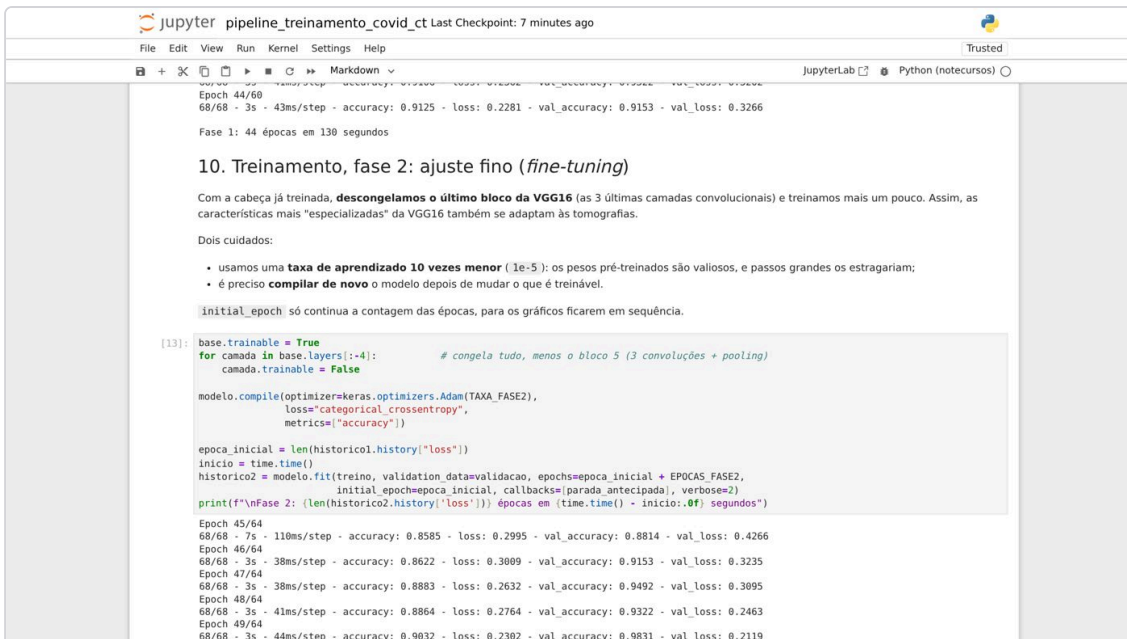
Antes de treinar, compilamos o modelo. O otimizador Adam ajusta os pesos; a função de perda mede o erro das previsões; e a acurácia é a porcentagem de acertos. O early stopping vigia a validação: se ela parar de melhorar por oito épocas, o treino para e os melhores pesos são restaurados.



```
Epoch 1/60
68/68 - 10s - 153ms/step - accuracy: 0.5605 - loss: 2.0178 - val_accuracy: 0.6780 - val_loss: 1.0002
Epoch 2/60
68/68 - 3s - 39ms/step - accuracy: 0.6313 - loss: 1.5389 - val_accuracy: 0.7458 - val_loss: 0.7476
Epoch 3/60
68/68 - 3s - 38ms/step - accuracy: 0.6350 - loss: 1.3239 - val_accuracy: 0.7966 - val_loss: 0.6758
Epoch 4/60
68/68 - 3s - 41ms/step - accuracy: 0.6927 - loss: 1.1382 - val_accuracy: 0.8305 - val_loss: 0.5594
Epoch 5/60
68/68 - 3s - 41ms/step - accuracy: 0.6927 - loss: 0.9624 - val_accuracy: 0.8475 - val_loss: 0.5673
Epoch 6/60
68/68 - 3s - 42ms/step - accuracy: 0.7263 - loss: 0.8023 - val_accuracy: 0.8475 - val_loss: 0.4782
Epoch 7/60
68/68 - 3s - 40ms/step - accuracy: 0.7263 - loss: 0.7786 - val_accuracy: 0.8983 - val_loss: 0.4097
Epoch 8/60
68/68 - 3s - 43ms/step - accuracy: 0.7281 - loss: 0.7419 - val_accuracy: 0.8983 - val_loss: 0.3902
Epoch 9/60
68/68 - 3s - 43ms/step - accuracy: 0.7467 - loss: 0.6709 - val_accuracy: 0.8983 - val_loss: 0.4215
Epoch 10/60
68/68 - 3s - 43ms/step - accuracy: 0.7672 - loss: 0.5949 - val_accuracy: 0.8814 - val_loss: 0.4117
Epoch 11/60
68/68 - 3s - 42ms/step - accuracy: 0.7691 - loss: 0.5689 - val_accuracy: 0.8814 - val_loss: 0.3981
Epoch 12/60
68/68 - 3s - 41ms/step - accuracy: 0.7877 - loss: 0.4852 - val_accuracy: 0.8814 - val_loss: 0.3842
Epoch 13/60
68/68 - 3s - 43ms/step - accuracy: 0.7840 - loss: 0.5366 - val_accuracy: 0.8814 - val_loss: 0.3702
Epoch 14/60
68/68 - 3s - 41ms/step - accuracy: 0.7747 - loss: 0.4929 - val_accuracy: 0.8814 - val_loss: 0.3802
Epoch 15/60
68/68 - 3s - 38ms/step - accuracy: 0.8007 - loss: 0.4659 - val_accuracy: 0.8983 - val_loss: 0.3857
Epoch 16/60
68/68 - 3s - 38ms/step - accuracy: 0.8026 - loss: 0.4410 - val_accuracy: 0.9153 - val_loss: 0.3491
Epoch 17/60
68/68 - 3s - 38ms/step - accuracy: 0.8194 - loss: 0.4234 - val_accuracy: 0.9153 - val_loss: 0.3528
Epoch 18/60
68/68 - 3s - 38ms/step - accuracy: 0.8361 - loss: 0.3547 - val_accuracy: 0.9153 - val_loss: 0.3553
Epoch 19/60
68/68 - 3s - 40ms/step - accuracy: 0.8212 - loss: 0.4093 - val_accuracy: 0.9153 - val_loss: 0.3666
Epoch 20/60
68/68 - 3s - 42ms/step - accuracy: 0.8399 - loss: 0.3752 - val_accuracy: 0.9153 - val_loss: 0.3367
Epoch 21/60
68/68 - 3s - 42ms/step - accuracy: 0.8361 - loss: 0.3288 - val_accuracy: 0.9153 - val_loss: 0.3678
Epoch 22/60
68/68 - 3s - 44ms/step - accuracy: 0.8287 - loss: 0.3678 - val_accuracy: 0.8983 - val_loss: 0.3901
```

Na tela: Linhas das épocas com loss e val_accuracy telas/18_treino1_inicio.png

Cada linha é uma época. A perda, o loss, vai caindo, e a acurácia na validação sobe rápido: de 68% na primeira época para cerca de 90% em poucas épocas.



```
Epoch 44/60
68/68 - 3s - 43ms/step - accuracy: 0.9125 - loss: 0.2281 - val_accuracy: 0.9153 - val_loss: 0.3266

Fase 1: 44 épocas em 130 segundos

10. Treinamento, fase 2: ajuste fino (fine-tuning)

Com a cabeça já treinada, descongelamos o último bloco da VGG16 (as 3 últimas camadas convolucionais) e treinamos mais um pouco. Assim, as características mais "especializadas" da VGG16 também se adaptam às tomografias.

Dois cuidados:



- usamos uma taxa de aprendizado 10 vezes menor ( $1e-5$ ): os pesos pré-treinados são valiosos, e passos grandes os estragariam;
- é preciso compilar de novo o modelo depois de mudar o que é treinável.

initial_epoch só continua a contagem das épocas, para os gráficos ficarem em sequência.

[13]: base.trainable = True
      for camada in base.layers[:4]: # congela tudo, menos o bloco 5 (3 convoluções + pooling)
          camada.trainable = False

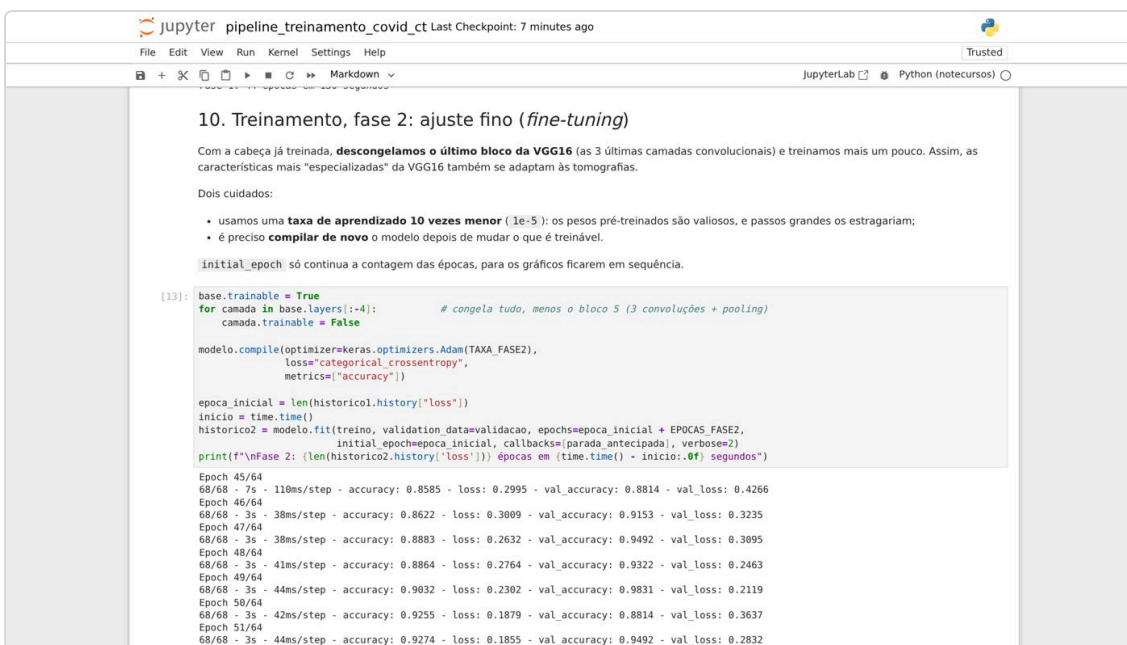
      modelo.compile(optimizer=keras.optimizers.Adam(TAXA_FASE2),
                    loss="categorical_crossentropy",
                    metrics=["accuracy"])

      epoca_inicial = len(historico1.history["loss"])
      inicio = time.time()
      historico2 = modelo.fit(treino, validation_data=validacao, epochs=epoca_inicial + EPOCAS_FASE2,
                             initial_epoch=epoca_inicial, callbacks=[parada_anticipada], verbose=2)
      print(f"Fase 2: {len(historico2.history['loss'])} épocas em {time.time() - inicio:.0f} segundos")

Epoch 45/64
68/68 - 7s - 110ms/step - accuracy: 0.8585 - loss: 0.2995 - val_accuracy: 0.8814 - val_loss: 0.4266
Epoch 46/64
68/68 - 3s - 38ms/step - accuracy: 0.8622 - loss: 0.3009 - val_accuracy: 0.9153 - val_loss: 0.3235
Epoch 47/64
68/68 - 3s - 38ms/step - accuracy: 0.8883 - loss: 0.2632 - val_accuracy: 0.9492 - val_loss: 0.3095
Epoch 48/64
68/68 - 3s - 41ms/step - accuracy: 0.8864 - loss: 0.2764 - val_accuracy: 0.9322 - val_loss: 0.2463
Epoch 49/64
68/68 - 3s - 44ms/step - accuracy: 0.9032 - loss: 0.2302 - val_accuracy: 0.9831 - val_loss: 0.2119
```

Na tela: 'Fase 1: 44 épocas em 130 segundos' telas/19_treino1_fim.png

O early stopping encerrou a primeira fase na época 44, em pouco mais de dois minutos na GPU.

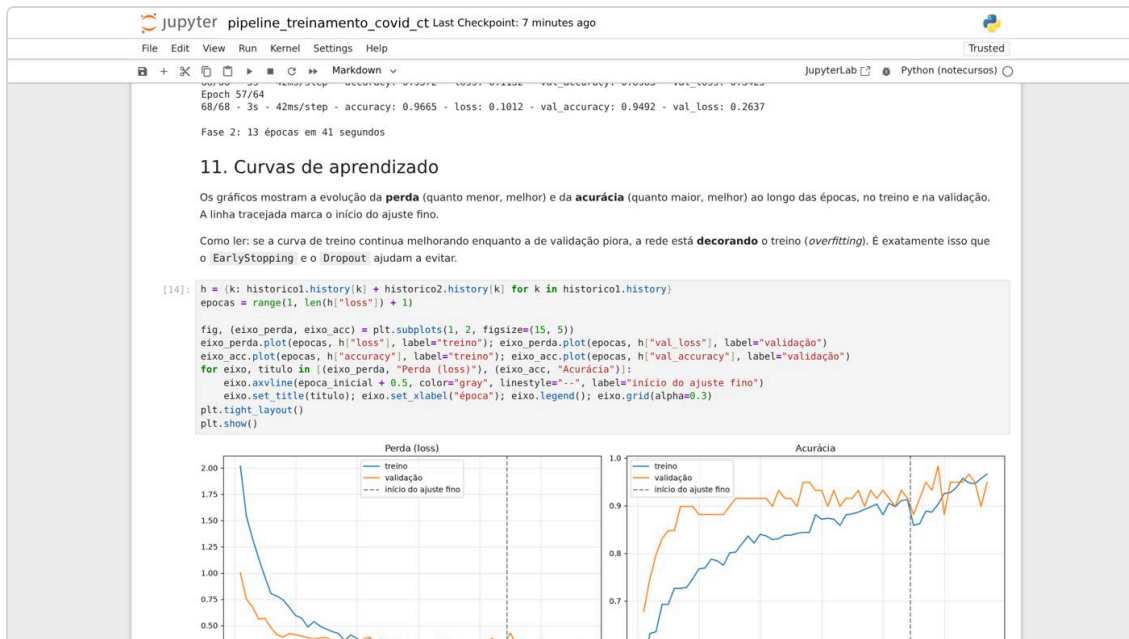


```
Epoch 45/64
68/68 - 7s - 110ms/step - accuracy: 0.8585 - loss: 0.2995 - val_accuracy: 0.8814 - val_loss: 0.4266
Epoch 46/64
68/68 - 3s - 38ms/step - accuracy: 0.8622 - loss: 0.3009 - val_accuracy: 0.9153 - val_loss: 0.3235
Epoch 47/64
68/68 - 3s - 38ms/step - accuracy: 0.8883 - loss: 0.2632 - val_accuracy: 0.9492 - val_loss: 0.3095
Epoch 48/64
68/68 - 3s - 41ms/step - accuracy: 0.8864 - loss: 0.2764 - val_accuracy: 0.9322 - val_loss: 0.2463
Epoch 49/64
68/68 - 3s - 44ms/step - accuracy: 0.9032 - loss: 0.2302 - val_accuracy: 0.9831 - val_loss: 0.2119
Epoch 50/64
68/68 - 3s - 42ms/step - accuracy: 0.9255 - loss: 0.1879 - val_accuracy: 0.8814 - val_loss: 0.3637
Epoch 51/64
68/68 - 3s - 44ms/step - accuracy: 0.9274 - loss: 0.1855 - val_accuracy: 0.9492 - val_loss: 0.2832

Fase 2: 20 épocas em 19 segundos
```

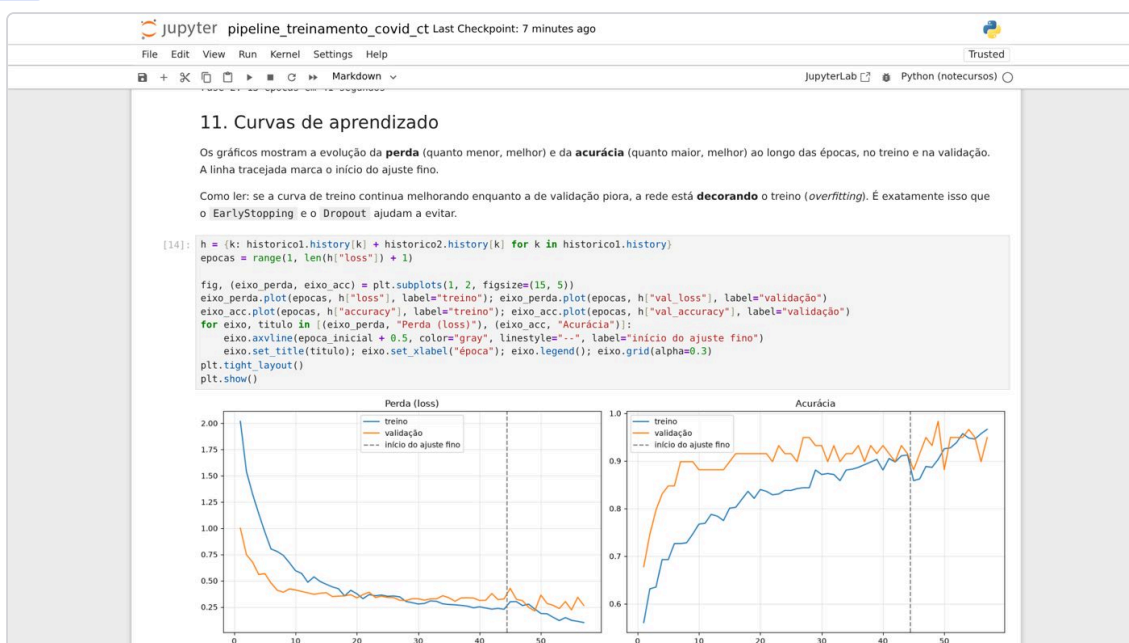
Na tela: Descongelando o bloco 5 com taxa $1e-5$ telas/20_ajustefino_explicacao.png

Na segunda fase, o ajuste fino, descongelamos o último bloco da VGG16 para que ele também se adapte às tomografias. Usamos uma taxa de aprendizado dez vezes menor, porque os pesos pré-treinados são valiosos e passos grandes os estragariam. E compilamos de novo o modelo.



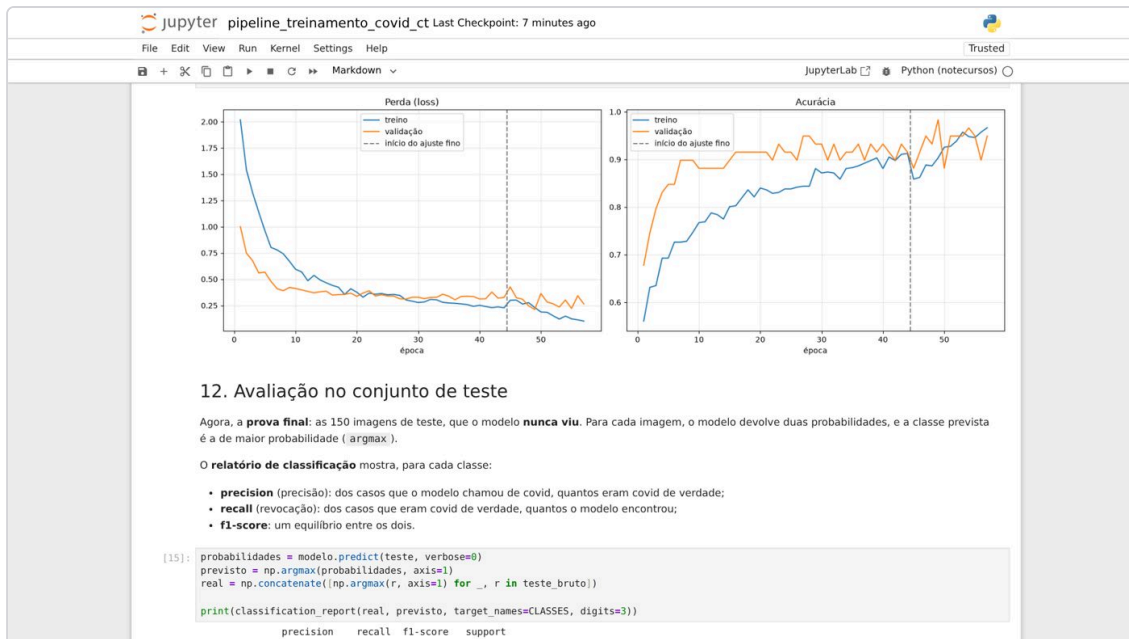
Na tela: 'Fase 2: 13 épocas em 41 segundos' telas/21_ajustefino_fim.png

O ajuste fino levou 13 épocas, em cerca de 40 segundos. A acurácia na validação chegou perto de 95%.



Na tela: Como ler as curvas e o código do gráfico telas/22_curvas_explicacao.png

As curvas de aprendizado mostram a evolução ao longo das épocas. Se o treino continua melhorando enquanto a validação piora, é sinal de overfitting.



Na tela: Perda e acurácia, com a linha do início do ajuste fino `telas/23_curvas_grafico.png`

À esquerda, a perda cai e se estabiliza; à direita, a acurácia sobe. A linha tracejada marca o início do ajuste fino, que ainda trouxe um ganho. Treino e validação andam juntos, sem sinal forte de overfitting.



Na tela: Relatório de classificação: acurácia 0,820 `telas/24_relatorio.png`

Agora, a prova final: as 150 imagens de teste, que o modelo nunca viu. O relatório mostra, para cada classe, a precisão, o recall e o f1. A acurácia geral foi de 82%, acima dos 73 a 77% dos notebooks originais.

Jupyter pipeline_treinamento_covid_ct Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Matriz de confusão, sensibilidade e especificidade

A **matriz de confusão** cruza a classe real (linhas) com a prevista (colunas). Na diagonal ficam os acertos; fora dela, os erros.

Considerando **covid** como o caso "positivo", a matriz tem quatro casos: **verdadeiros positivos** (VP: covid detectado), **falsos negativos** (FN: covid que passou como normal), **verdadeiros negativos** (VN: normal reconhecido) e **falsos positivos** (FP: normal acusado de covid). Com eles calculamos:

Métrica	Fórmula	Pergunta que responde
Recall (revocação) = Sensibilidade	$VP / (VP + FN)$	Dos pacientes com COVID-19, quantos o modelo detectou?
Especificidade	$VN / (VN + FP)$	Dos pacientes sem COVID-19, quantos foram reconhecidos como normais?
Precisão	$VP / (VP + FP)$	Quando o modelo diz "covid", quantas vezes acerta?
Acurácia	$(VP + VN) / \text{total}$	Quantos acertos no total?

Em diagnóstico médico, o **recall** (**sensibilidade**) costuma ser a métrica mais importante: um recall baixo significa casos de covid deixados passar (**falsos negativos**), o erro mais perigoso. Repare que, no relatório de classificação acima, o **recall** da linha **covid** é a sensibilidade e o **recall** da linha **normal** é a especificidade.

Para comparação: os notebooks originais deste projeto chegavam a **73% a 77% de acurácia**. Os números podem variar alguns pontos a cada execução, porque o treino na GPU tem uma parte aleatória (ordem das imagens, aumento de dados, *dropout*).

```
[16]: cm = confusion_matrix(real, previsto)
ConfusionMatrixDisplay(cm, display_labels=CLASSES).plot(cmap="Blues", colorbar=False)
plt.title("Matriz de confusão (conjunto de teste)")
plt.xlabel("classe prevista"); plt.ylabel("classe real")
plt.show()

VP, FN = cm[0, 0], cm[0, 1] # linha "covid": detectados / deixados passar
FP, VN = cm[1, 0], cm[1, 1] # linha "normal": alarmes falsos / reconhecidos

print(f"VP={VP} FN={FN} FP={FP} VN={VN}\n")
print(f"Recall (sensibilidade): {VP / (VP + FN):.1%} ({VP} de {VP + FN} casos de covid detectados)")
print(f"Especificidade: {VN / (VN + FP):.1%} ({VN} de {VN + FP} casos normais reconhecidos)")
print(f"Precisão: {VP / (VP + FP):.1%} ({VP} de {VP + FP} alertas de covid estavam certos)")
print(f"Acurácia: {(VP + VN) / cm.sum():.1%} ({VP + VN} de {cm.sum()} imagens)")
```

Matriz de confusão (conjunto de teste)

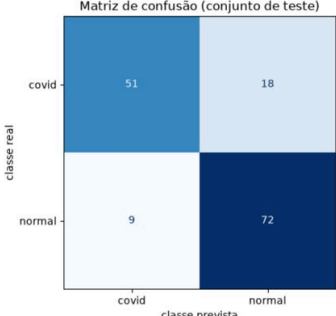
Na tela: Tabela de métricas com as fórmulas telas/25_metricas_explicacao.png

Considerando covid como o caso positivo, temos verdadeiros positivos, falsos negativos, verdadeiros negativos e falsos positivos. O recall, ou sensibilidade, mede quantos casos de covid o modelo detectou. A especificidade, quantos normais ele reconheceu. E a precisão, quantas vezes acertou ao dizer covid. Em medicina, o recall é crucial: um falso negativo é um paciente doente mandado para casa.

Jupyter pipeline_treinamento_covid_ct Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Matriz de confusão (conjunto de teste)



	covid	normal
covid	51	18
normal	9	72

VP=51 FN=18 FP=9 VN=72

Recall (sensibilidade): 73.9% (51 de 69 casos de covid detectados)
 Especificidade: 88.9% (72 de 81 casos normais reconhecidos)
 Precisão: 85.0% (51 de 60 alertas de covid estavam certos)
 Acurácia: 82.0% (123 de 150 imagens)

13. Salvando o modelo treinado

O modelo treinado é salvo num único arquivo `.keras`, com a arquitetura e os pesos. Assim ele pode ser usado depois, por exemplo numa aplicação web, **sem precisar treinar de novo**. Para conferir, carregamos o arquivo e verificamos que ele dá as mesmas previsões.

```
[17]: PASTA_MODELOS.mkdir(exist_ok=True)
caminho_modelo = PASTA_MODELOS / "covid_ct_vgg16.keras"
modelo.save(caminho_modelo)
```

Na tela: Matriz 51 / 18 / 9 / 72 telas/26_matriz_confusao.png

Na matriz de confusão, a diagonal são os acertos: 51 casos de covid detectados e 72 normais reconhecidos. Os erros: 18 casos de covid que passaram como normais e 9 normais acusados de covid.

pipeline_treinamento_covid_ct Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Markdown Python (notecursos)

covid normal

classe prevista

VP=51 FN=18 FP=9 VN=72

Recall (sensibilidade): 73.9% (51 de 69 casos de covid detectados)

Especificidade: 88.9% (72 de 81 casos normais reconhecidos)

Precisão: 85.0% (51 de 60 alertas de covid estavam certos)

Acurácia: 82.0% (123 de 150 imagens)

13. Salvando o modelo treinado

O modelo treinado é salvo num único arquivo `.keras`, com a arquitetura e os pesos. Assim ele pode ser usado depois, por exemplo numa aplicação web, **sem precisar treinar de novo**. Para conferir, carregamos o arquivo e verificamos que ele dá as mesmas previsões.

```
[17]: PASTA_MODELOS.mkdir(exist_ok=True)
caminho_modelo = PASTA_MODELOS / "covid_ct_vgg16.keras"
modelo.save(caminho_modelo)
print(f"Modelo salvo em {caminho_modelo} ((caminho_modelo.stat().st_size / 1e6:.0f) MB)")

modelo_carregado = keras.models.load_model(caminho_modelo)
iguais = np.array_equal(np.argmax(modelo_carregado.predict(teste, verbose=0), axis=1), previsto)
print("Modelo carregado dá as mesmas previsões:", iguais)

Modelo salvo em modelos/covid_ct_vgg16.keras (117 MB)
Modelo carregado dá as mesmas previsões: True
```

14. Usando o modelo: previsão de uma tomografia

Por fim, o uso prático: uma função que recebe o caminho de **uma** tomografia e devolve o diagnóstico do modelo com o **grau de certeza** (a probabilidade da classe escolhida), como nos notebooks originais.

A imagem passa pelas **mesmas transformações** do treino: redimensionamento para 224x224 e `preprocess_input`. Esquecer isso na hora da previsão é outro erro comum.

```
[18]: def prever_imagem(caminho, modelo=modelo_carregado):
    imagem = keras.utils.load_img(caminho, target_size=TAMANHO_IMAGEM, color_mode="rgb")
    lote = preprocess_input(np.expand_dims(keras.utils.img_to_array(imagem), axis=0))
    prob = modelo.predict(lote, verbose=0)[0]
    classe = CLASSES[int(np.argmax(prob))]
    resultado = "COVID-19 POSITIVO" if classe == "covid" else "COVID-19 NEGATIVO (normal)"
    return resultado, float(np.max(prob)), imagem

for caminho in [teste_bruto.file_paths[0], teste_bruto.file_paths[-1]]:
    resultado, certeza, _ = prever_imagem(caminho)
    print(f"(Path(caminho).parent.name: {Path(caminho).name[:40]:<40}) → (resultado) (certeza: {certeza:.1%})")

covid/2020_01_24_919183-p27-132.png → COVID-19 POSITIVO (certeza: 94.1%)
```

Na tela: Recall 73,9%, especificidade 88,9%, precisão 85%, acurácia 82%

telas/27_metricas_valores.png

Em números: recall de 73,9%, especificidade de 88,9%, precisão de 85% e acurácia de 82%. O modelo é mais seguro para descartar do que para detectar a doença, e melhorar o recall seria o próximo objetivo.

pipeline_treinamento_covid_ct Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

Markdown Python (notecursos)

13. Salvando o modelo treinado

O modelo treinado é salvo num único arquivo `.keras`, com a arquitetura e os pesos. Assim ele pode ser usado depois, por exemplo numa aplicação web, **sem precisar treinar de novo**. Para conferir, carregamos o arquivo e verificamos que ele dá as mesmas previsões.

```
[17]: PASTA_MODELOS.mkdir(exist_ok=True)
caminho_modelo = PASTA_MODELOS / "covid_ct_vgg16.keras"
modelo.save(caminho_modelo)
print(f"Modelo salvo em {caminho_modelo} ((caminho_modelo.stat().st_size / 1e6:.0f) MB)")

modelo_carregado = keras.models.load_model(caminho_modelo)
iguais = np.array_equal(np.argmax(modelo_carregado.predict(teste, verbose=0), axis=1), previsto)
print("Modelo carregado dá as mesmas previsões:", iguais)

Modelo salvo em modelos/covid_ct_vgg16.keras (117 MB)
Modelo carregado dá as mesmas previsões: True
```

14. Usando o modelo: previsão de uma tomografia

Por fim, o uso prático: uma função que recebe o caminho de **uma** tomografia e devolve o diagnóstico do modelo com o **grau de certeza** (a probabilidade da classe escolhida), como nos notebooks originais.

A imagem passa pelas **mesmas transformações** do treino: redimensionamento para 224x224 e `preprocess_input`. Esquecer isso na hora da previsão é outro erro comum.

```
[18]: def prever_imagem(caminho, modelo=modelo_carregado):
    imagem = keras.utils.load_img(caminho, target_size=TAMANHO_IMAGEM, color_mode="rgb")
    lote = preprocess_input(np.expand_dims(keras.utils.img_to_array(imagem), axis=0))
    prob = modelo.predict(lote, verbose=0)[0]
    classe = CLASSES[int(np.argmax(prob))]
    resultado = "COVID-19 POSITIVO" if classe == "covid" else "COVID-19 NEGATIVO (normal)"
    return resultado, float(np.max(prob)), imagem

for caminho in [teste_bruto.file_paths[0], teste_bruto.file_paths[-1]]:
    resultado, certeza, _ = prever_imagem(caminho)
    print(f"(Path(caminho).parent.name: {Path(caminho).name[:40]:<40}) → (resultado) (certeza: {certeza:.1%})")

covid/2020_01_24_919183-p27-132.png → COVID-19 POSITIVO (certeza: 94.1%)
```

Na tela: Modelo salvo (117 MB) e carregado com as mesmas previsões telas/28_salvar_modelo.png

O modelo treinado é salvo num único arquivo ponto keras, com a arquitetura e os pesos, e pode ser usado depois sem treinar de novo. Carregamos o arquivo e conferimos: as previsões são as mesmas.

```

14. Usando o modelo: previsão de uma tomografia

Por fim, o uso prático: uma função que recebe o caminho de uma tomografia e devolve o diagnóstico do modelo com o grau de certeza (a probabilidade da classe escolhida), como nos notebooks originais.

A imagem passa pelas mesmas transformações do treino: redimensionamento para 224x224 e preprocess_input. Esquecer isso na hora da previsão é outro erro comum.

[18]: def prever_imagem(caminho, modelo=modelo_carregado):
        imagem = keras.utils.load_img(caminho, target_size=TAMANHO_IMAGEM, color_mode="rgb")
        lote = preprocess_input(np.expand_dims(keras.utils.img_to_array(imagem), axis=0))
        prob = modelo.predict(lote, verbose=0)[0]
        classe = CLASSES[int(np.argmax(prob))]
        resultado = "COVID-19 POSITIVO" if classe == "covid" else "COVID-19 NEGATIVO (normal)"
        return resultado, float(np.max(prob)), imagem

        for caminho in (teste_bruto.file_paths[0], teste_bruto.file_paths[-1]):
            resultado, certeza, _ = prever_imagem(caminho)
            print(f'{Path(caminho).parent.name[:40]:<40} -> (resultado) (certeza: {certeza:.1%})')

            covid/2020.01.24.919183-p27-132.png -> COVID-19 POSITIVO (certeza: 94.1%)
            normal/997.png -> COVID-19 NEGATIVO (normal) (certeza: 100.0%)

        Para visualizar melhor, sorteamos 8 tomografias do conjunto de teste. O título mostra a classe real e a previsão do modelo: verde quando ele acertou,
        vermelho quando errou.

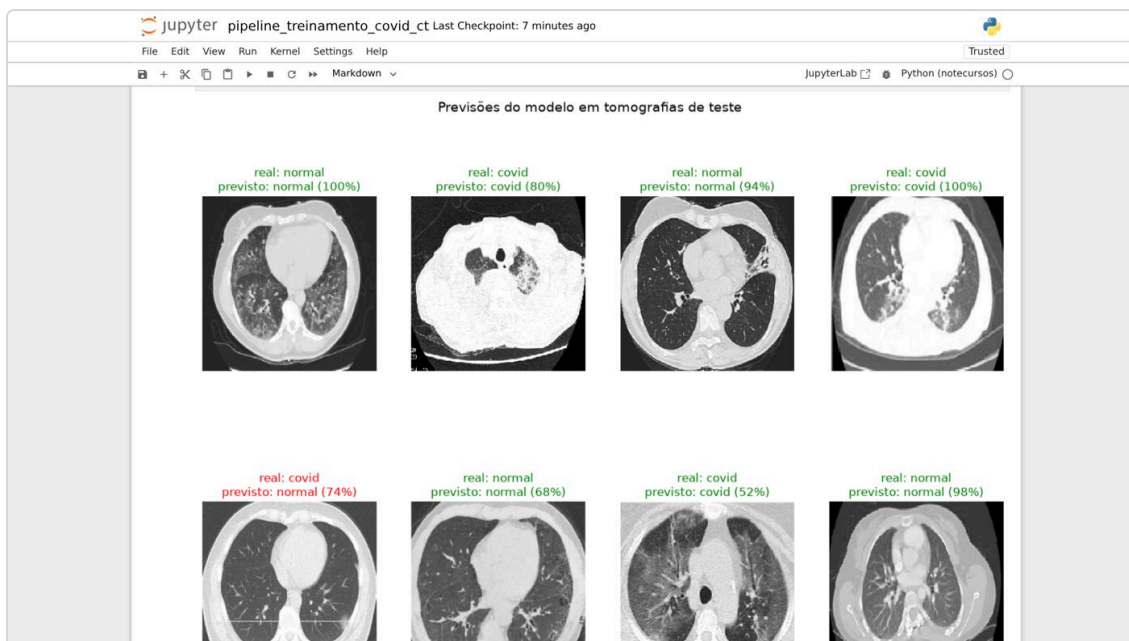
[19]: amostra = random.Random(7).sample(teste_bruto.file_paths, 8)
        fig, eixos = plt.subplots(2, 4, figsize=(15, 10), gridspec_kw={"hspace": 0.35})
        for eixo, caminho in zip(eixos.flat, amostra):
            resultado, certeza, imagem = prever_imagem(caminho)
            real_classe = Path(caminho).parent.name
            acertou = (real_classe == "covid") == resultado.startswith("COVID-19 POSITIVO")
            eixo.imshow(imagem); eixo.axis("off")
            eixo.set_title(f'real: {real_classe}\nprevisto: {"covid" if resultado.startswith("COVID-19 POSITIVO") else "normal"} ({certeza:.0%})',
                           color="green" if acertou else "red")

        plt.suptitle("Previsões do modelo em tomografias de teste", fontsize=14)
        plt.show()
  
```

Previsões do modelo em tomografias de teste

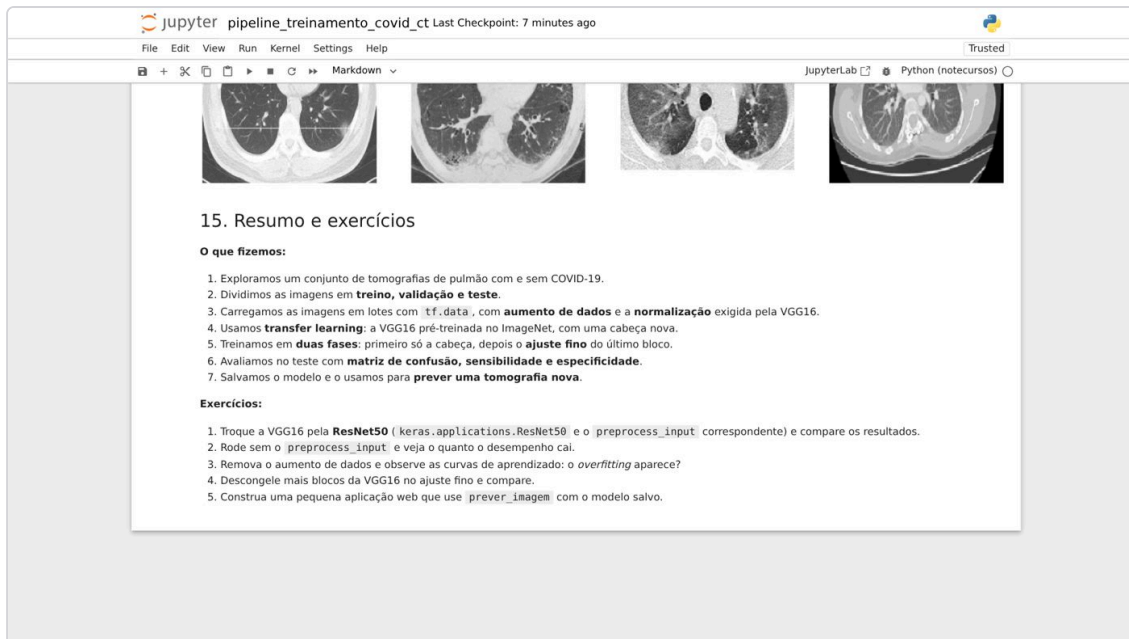
Na tela: Função prever_imagem e os resultados 94,1% e 100% telas/29_previsao_imagem.png

Por fim, o uso prático: uma função que recebe uma tomografia e devolve o diagnóstico com o grau de certeza. A imagem passa pelas mesmas transformações do treino. Nos exemplos, uma tomografia de covid foi classificada como positiva, com 94% de certeza, e uma normal, como negativa.



Na tela: Grade de 8 previsões: verde acerto, vermelho erro telas/30_previsoes_grade.png

Nesta grade, verde é acerto e vermelho é erro. O modelo acertou sete de oito. O erro foi um caso de covid classificado como normal, justamente o tipo de erro mais perigoso. Repare também num acerto com só 52% de certeza: o modelo estava em dúvida.



jupyter pipeline_treinamento_covid_ct Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python (notecursos)

15. Resumo e exercícios

O que fizemos:

1. Exploramos um conjunto de tomografias de pulmão com e sem COVID-19.
2. Dividimos as imagens em **treino, validação e teste**.
3. Carregamos as imagens em lotes com `tf.data`, com **aumento de dados** e a **normalização** exigida pela VGG16.
4. Usamos **transfer learning**: a VGG16 pré-treinada no ImageNet, com uma cabeça nova.
5. Treinamos em **duas fases**: primeiro só a cabeça, depois o **ajuste fino** do último bloco.
6. Avaliamos no teste com **matriz de confusão, sensibilidade e especificidade**.
7. Salvamos o modelo e o usamos para **prever uma tomografia nova**.

Exercícios:

1. Troque a VGG16 pela **ResNet50** (`keras.applications.ResNet50` e o `preprocess_input` correspondente) e compare os resultados.
2. Rode sem o `preprocess_input` e veja o quanto o desempenho cai.
3. Remova o aumento de dados e observe as curvas de aprendizado: o *overfitting* aparece?
4. Descongele mais blocos da VGG16 no ajuste fino e compare.
5. Construa uma pequena aplicação web que use `prever_imagem` com o modelo salvo.

Na tela: Lista do que fizemos e os 5 exercícios `telas/31_resumo_exercicios.png`

Recapitulando: exploramos os dados, dividimos em treino, validação e teste, aplicamos aumento de dados e normalização, usamos transfer learning com a VGG16, treinamos em duas fases e avaliamos com recall e especificidade. Como exercício, troquem a VGG16 pela ResNet50, rodem sem a normalização e comparem. Até a próxima!